

20250708 日报

今日工作内容

1. 研究七彩石如果故障，该怎么让容器还是能成功启动呢？=>备用“七彩石”策略，主要思路有三种：

1、根据拉取配置失败次数来自动控制读取七彩石配置的备用开关，比如：失败 10 次后就切换到 reids 读取配置，而不从七彩石读：

- **故障检测机制：**以七彩石（配置中心）拉取配置超时作为其服务不可用的核心判据。当应用在预设时间内无法从七彩石获取配置时，即判定为服务故障。通过多次超时判断，来避免短暂网络波动误判；

- **降级开关策略：**引入阈值参数 `ConnectrainbowfailedNumbersLimit` 作为降级开关的触发条件。该参数定义容器启动时允许的七彩石连接失败次数上限。若实际失败次数 未达到阈值，应用仍尝试从七彩石获取配置；达到阈值后，自动切换至备用配置源（Redis）；

- **自定义七彩石钩子函数逻辑：**注册自定义钩子函数监听七彩石插件拉取失败事件（这会覆盖自动 `trpc-go` 框架下的钩子函数）。当插件抛出超时异常时，触发钩子函数执行以下逻辑：

- 对 Redis 中的计数器键 `Connectrainbowfailednumberskey` 执行原子自增操作（INCR）；

- 校验计数器值是否达到 `ConnectrainbowfailedNumbersLimit` 阈值，若达到则切换配置源；

- **降级生效条件：**当 `Connectrainbowfailednumberskey` 的计数值 $\geq$ `ConnectrainbowfailedNumbersLimit` 时，从 Redis 读取配置而非七彩石；

- **渐进式状态回退：**降级触发后，将计数器值自减 1（DECR），而非重置为 0。此举确保后续容器启动时仅需经历 `ConnectrainbowfailedNumbersLimit - N` 次失败即可再次触发降级（N 为已回退次数），避免反复触发容器重启风暴；

Q：为什么要进行回退，不回退的话下一个容器启动不就不会重启了吗？

A：进行回退是因为如果故障恢复了，这样就可以立马自动发现。

- **冷启动：**若首个容器启动时七彩石正常，则会自动拉取配置并写入 Redis，无需手

动干预；若七彩石预故障，需手动初始化 Redis 配置，确保降级兜底可用；

- **数据同步：**七彩石配置变更时，同步写入 Redis 确保备用数据时效性；

优点：

- 自动切换，无需手动控制；
- redis 中的信息是自动实时更新的；
- 维护简单，因为配置自动更新、故障自动切换；
- 七彩石如果故障恢复，新起的容器可以立马切换回从七彩石中读；

缺点：

- 代码解耦度不好，需要在 rainbow 初始化函数中 copy 切换代码；
- 配置信息如果不准，不容易发现，需要自己手动去 redis 中查看；
- 如果七彩石在下一个容器启动时仍是故障，会因为前一个容器的渐进式回退导致仍自己会重启几次才会从 redis 中读；

主要逻辑代码：

- **autorainbow/autoconfig.go：**

```

type Config struct {
    Name string `yaml:"name"`
    Age  int    `yaml:"age"`
}

var gCfg atomic.Value

func Init() {
    ctx := context.Background()
    connectrainbowfailednumbers, err :=
redis.Int(proxies.StandbyRainbowRedis.Do(context.Background(),
"GET", tools.Connectrainbowfailednumberskey))
    log.Info(connectrainbowfailednumbers)
    if err != nil {
        log.Fatal(err)
    }
    if err == redis.ErrNil && connectrainbowfailednumbers == 0
{ // 键不存在
        expose(ctx) // 兜底
    }
    if connectrainbowfailednumbers >=
tools.ConnectrainbowfailedNumbersLimit { // 连接失败次数超过阈值，从
redis 中获取配置
        val, err := getFromLocalRainbow(ctx)
        if err != nil {
            log.Fatal(err)
        }
        cfg := getDefaultConfig()
        if err := yaml.Unmarshal(stringtoBytes(val), cfg); err !=
nil {
            log.Error(err)
            metrics.Counter("config-更新失败").Incr()
        } else {
            log.Infof("%+v", cfg)
            gCfg.Store(cfg)
            metrics.Counter("config-更新成功").Incr()
        }
        decrCounter(ctx, tools.Connectrainbowfailednumberskey) //
自减计数器
    } else { // 七彩石正常获取配置
        val, err := config.Get("rainbow").Watch(ctx, "config")
        if err != nil {

```

● autorainbow/init.go

```
func init() {
    ctx := context.Background()
    // 注册钩子函数覆盖原有 trpc-go 框架中的逻辑
    plugin.RegisterSetupHook("config-rainbow", func(setup func()
error) error {
    ch := make(chan error)
    go func() { ch <- setup() }()
    select {
    case err := <-ch:
        return err // 正常完成, 返回错误或 nil
    case <-time.After(plugin.SetupTimeout):
        log.Info("timeout callback")
        timeoutCallback(ctx,
tools.Connectrainbowfailednumberskey) // 调用超时回调函数
    }
    return nil
})
}

func timeoutCallback(ctx context.Context,
connectrainbowfailednumberskey string) {
    _, err := proxies.StandbyRainbowRedis.Do(ctx, "INCR",
connectrainbowfailednumberskey)
    if err != nil {
        log.Fatal(err)
    }
}
```

● tools/utls.go

```
var ConnectrainbowfailedNumbersLimit = 10 // 触发切换到 redis 读取七彩石配置的开关阈值

var Connectrainbowfailednumberskey = "failurenumbers" // redis 中对应连接七彩石失败次数的 key
```

- proxies/redis_client.go

```
var StandbyRainbowRedis =
redis.NewClientProxy("trpc.redis.standbyrainbowredis.service",
    client.WithTarget("ip://127.0.0.1:6379"))
```

自测结果：假设首次拉取配置成功，然后在本机模拟拉取配置失败 10 次，验证第 11 次它是否切换到了从 redis 中拉取，并拉取成功。

- 假设首次从七彩石拉取配置成功，则将配置信息写入了 redis：

```
127.0.0.1:6379> get config
"Age: 25\nName: drewjjli\n"
127.0.0.1:6379>
```

- 后续 10 次拉取配置失败，此时会在 redis 中新增一个计数器键值对，对应 key="failurenumbers" 的值应该为 10

```
开始编译Go程序...
编译成功！生成可执行文件：server/server
启动服务...
2025/07/08 20:25:07 ip.go:125: [debug] set locap ip from server: 30.163.240.131:8080
2025/07/08 20:25:07 ip.go:144: [info] local ip: 21.6.68.166
2025-07-08 20:25:07.097 INFO autorainbow/main.go:23 timeout callback
2025-07-08 20:25:07.101 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-07-08 20:25:07.102 DEBUG debugLog@v0.1.13/log.go:229 client request:trpc.redis.standbyrainbowredis.service/GET, cost:150.012µs, to:127.0.0.1:6379
panic: runtime error: invalid memory address or nil pointer dereference
[signal SIGSEGV: segmentation violation code=0x1 addr=0x38 pc=0xe9327c]

goroutine 1 [running]:
git.woa.com/drewjjli/test/autorainbow.Init()
/data/workspace/test/autorainbow/autoconfig.go:49 +0x3dc
main.main()
/data/workspace/test/main.go:23 +0x33
```

```
127.0.0.1:6379> keys *
1) "failurenumbers"
2) "config"
3) "timeout"
127.0.0.1:6379> get failurenumbers
"10"
```

- 第 11 次启动服务，就可以成功拉起配置了，此时就是从 redis 中读取：

```
开始编译Go程序...
编译成功！生成可执行文件：server/server
启动服务...
2025/07/08 20:26:58 ip.go:125: [debug] set locap ip from server: 30.163.240.131:8080
2025/07/08 20:26:58 ip.go:144: [info] local ip: 21.6.68.166
2025-07-08 20:26:58.610 INFO autorainbow/main.go:23 timeout callback
2025-07-08 20:26:58.611 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-07-08 20:26:58.612 DEBUG debugLog@v0.1.13/log.go:229 client request:trpc.redis.standbyrainbowredis.service/GET, cost:400.201µs, to:127.0.0.1:6379
2025-07-08 20:26:58.613 DEBUG debugLog@v0.1.13/log.go:229 client request:trpc.redis.standbyrainbowredis.service/GET, cost:312.760µs, to:127.0.0.1:6379
2025-07-08 20:26:58.613 INFO autorainbow/autoconfig.go:43 &{Name:drewjjli Age:25}
2025-07-08 20:26:58.614 DEBUG debugLog@v0.1.13/log.go:229 client request:trpc.redis.standbyrainbowredis.service/DECR, cost:118.235µs, to:127.0.0.1:6379
2025-07-08 20:26:58.614 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1
2025-07-08 20:26:58.614 INFO server/service.go:207 process: 1628681, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8080, serving ...
2025-07-08 20:26:58.614 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

- 需要人为发现故障发生和故障恢复;

主要代码逻辑:

- [handrainbow/handrainbow.go](https://github.com/handrainbow/handrainbow.go)

```

type Config struct {
    Name string `yaml:"name"`
    Age  int    `yaml:"age"`
}

var gCfg atomic.Value

// Init 初始化配置
func Init() {
    val, err := getFromRedis(context.Background())
    if err != nil {
        log.Fatal(err)
    }
    cfg := getDefaultConfig()
    if err := yaml.Unmarshal(stringtoBytes(val), cfg); err != nil
    {
        log.Error(err)
        metrics.Counter("config-更新失败").Incr()
    } else {
        log.Infof("%+v", cfg)
        gCfg.Store(cfg)
        metrics.Counter("config-更新成功").Incr()
    }
}

func stringtoBytes(s string) []byte {
    return []byte(s)
}

// 从 redis 中获取备用 rainbow 配置
func getFromRedis(ctx context.Context) (string, error) {
    val, err := redis.String(proxyes.StandbyRainbowRedis.Do(ctx,
    "GET", "config"))
    if err != nil {
        log.Fatal("first operation failed, the file does not
    exist") // 读取配置失败
        return "", err
    }
    return val, nil
}

func getDefaultConfig() *Config {

```

- rainbow/config.go

```
// 只需要在原始的 rainbow 初始化中添加同步写 redis  
proxies.StandbyRainbowRedis.Do(context.Background(), "SET",  
"config", yamldata)
```

直接在主 main()中调用 handrainbow.Init()和 cfg := handrainbow.GetConfig()即可，其结果如下：

```
[root@drewjjli-it@osepea test]# ./server.sh  
开始编译Go程序...  
编译成功! 生成可执行文件: server/server  
启动服务...  
2025/07/08 20:50:27 ip.go:125: [debug] set locap ip from server: 30.163.240.131:8080  
2025/07/08 20:50:27 ip.go:144: [info] local ip: 21.6.68.166  
2025-07-08 20:50:27.248 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota  
2025-07-08 20:50:27.249 DEBUG debuglog@v0.1.13/log.go:229 client request:/trpc.redis.standbyrainbowredis.service/GET, cost:341.692µs, to:127.0.0.1:6379  
2025-07-08 20:50:27.249 INFO handrainbow/handconfig.go:32 &{Name:drewjjli Age:25}  
&{drewjjli 25}  
2025-07-08 20:50:27.249 INFO tnet/server_transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1  
2025-07-08 20:50:27.250 INFO server/service.go:207 process: 1654764, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8080, serving ...  
2025-07-08 20:50:27+08:00 INFO tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

3、不用代码控制切换逻辑，不用 redis，只需在 gdp 平台部署七彩石配置信息的备用 yaml 文件，然后手动在 gdp 新增挂载点即可：

此方法还未试验。

优点：

- 代码简单；

缺点：

- 维护不容易，因为需要手动将配置信息添加至 yaml 配置中，并且需要手动更新；
- 需要人为发现故障发生和故障恢复；

明日待办：

1. 上述的方案三的试验。